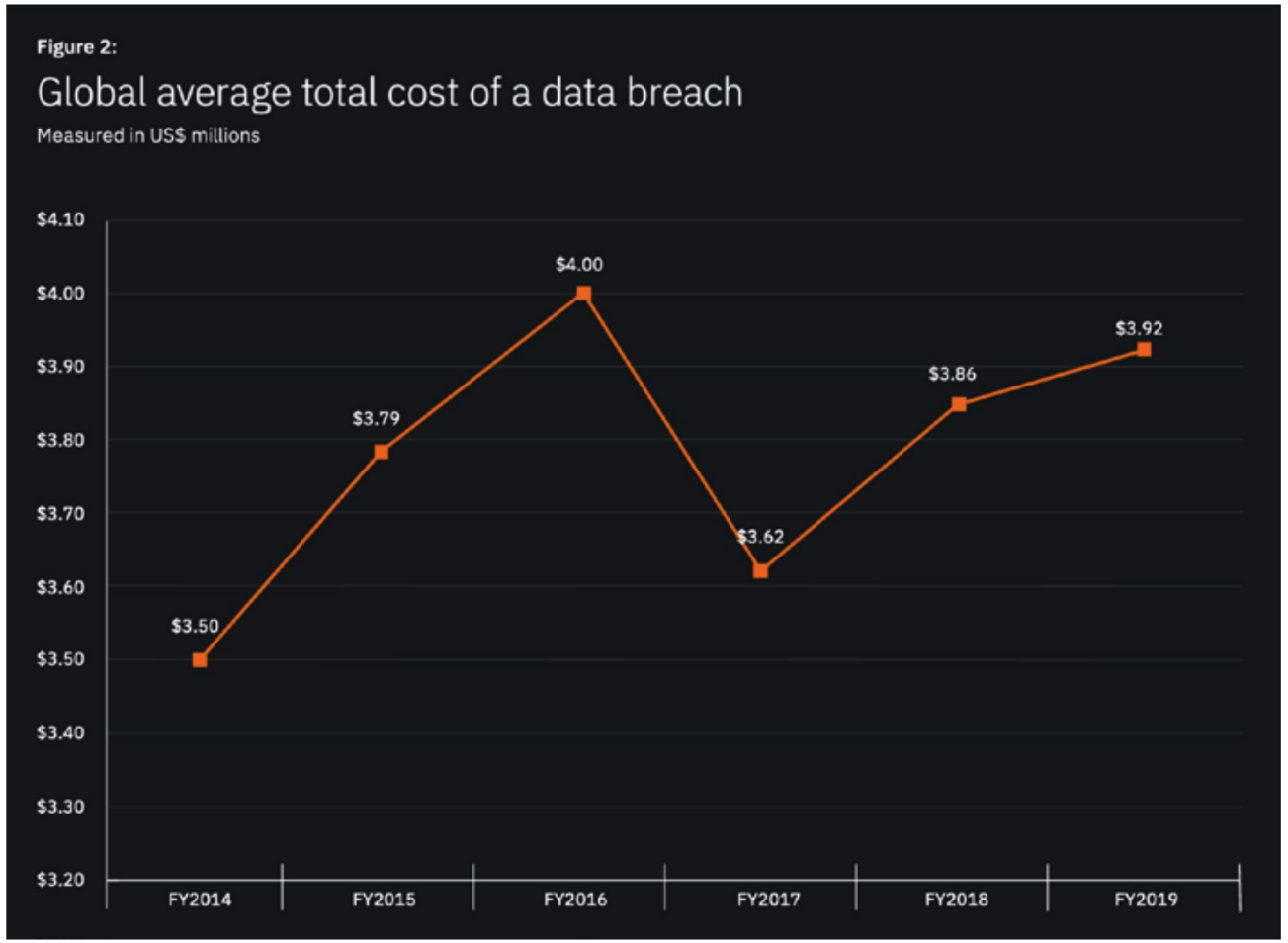
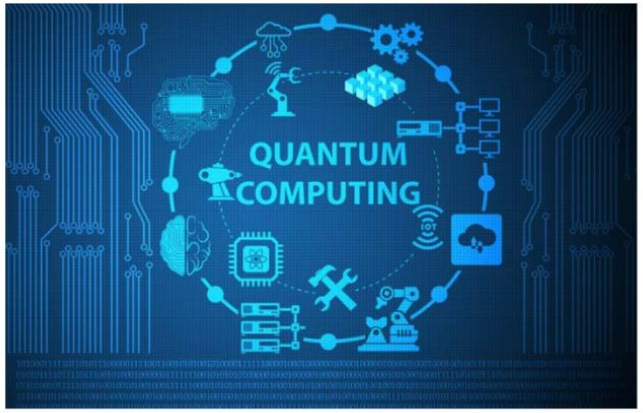
DevSecOps

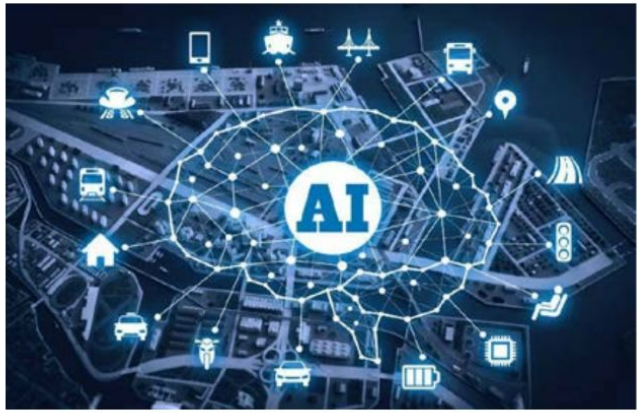
Data breach



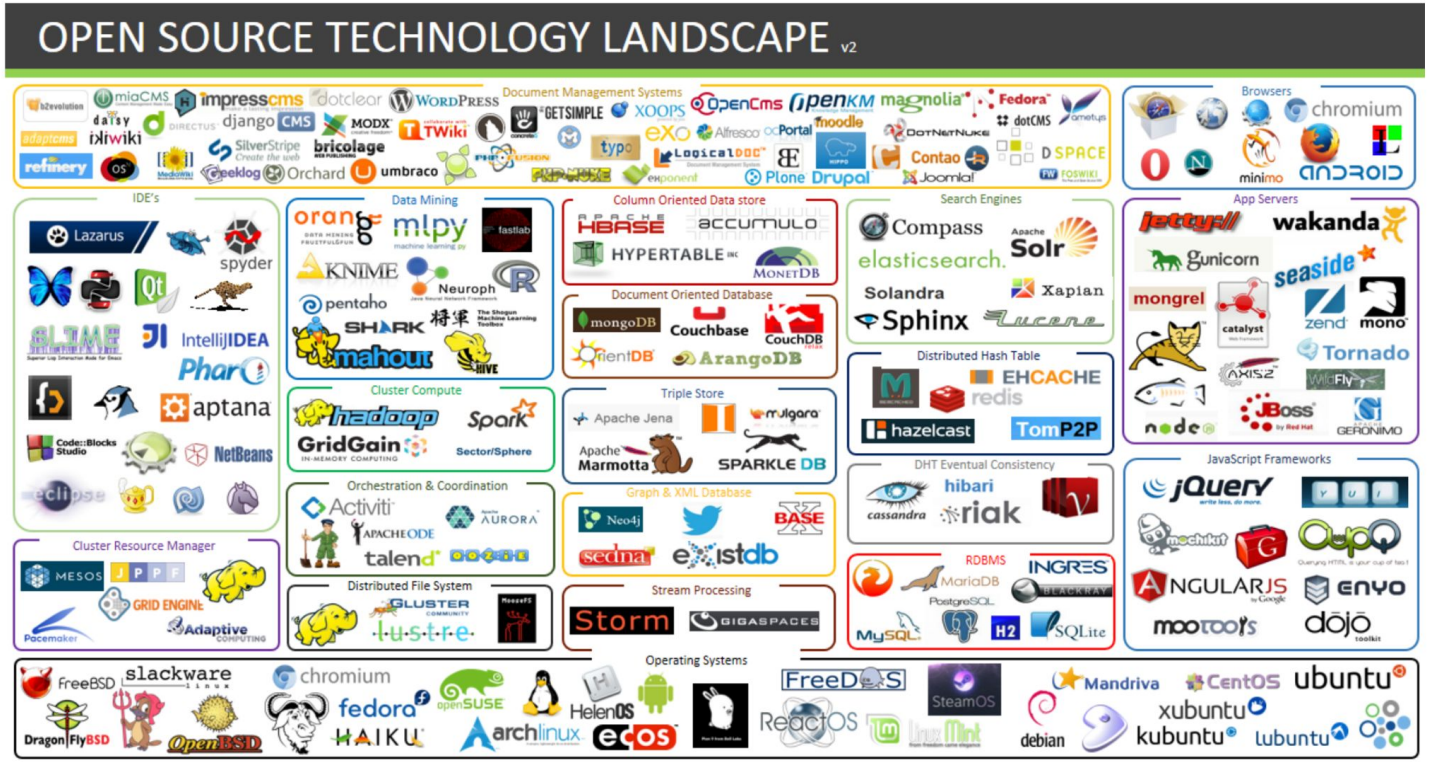
Source - IBM and the Ponemon Institute's annual Cost of a Data Breach_report

Threat to emerging technologies

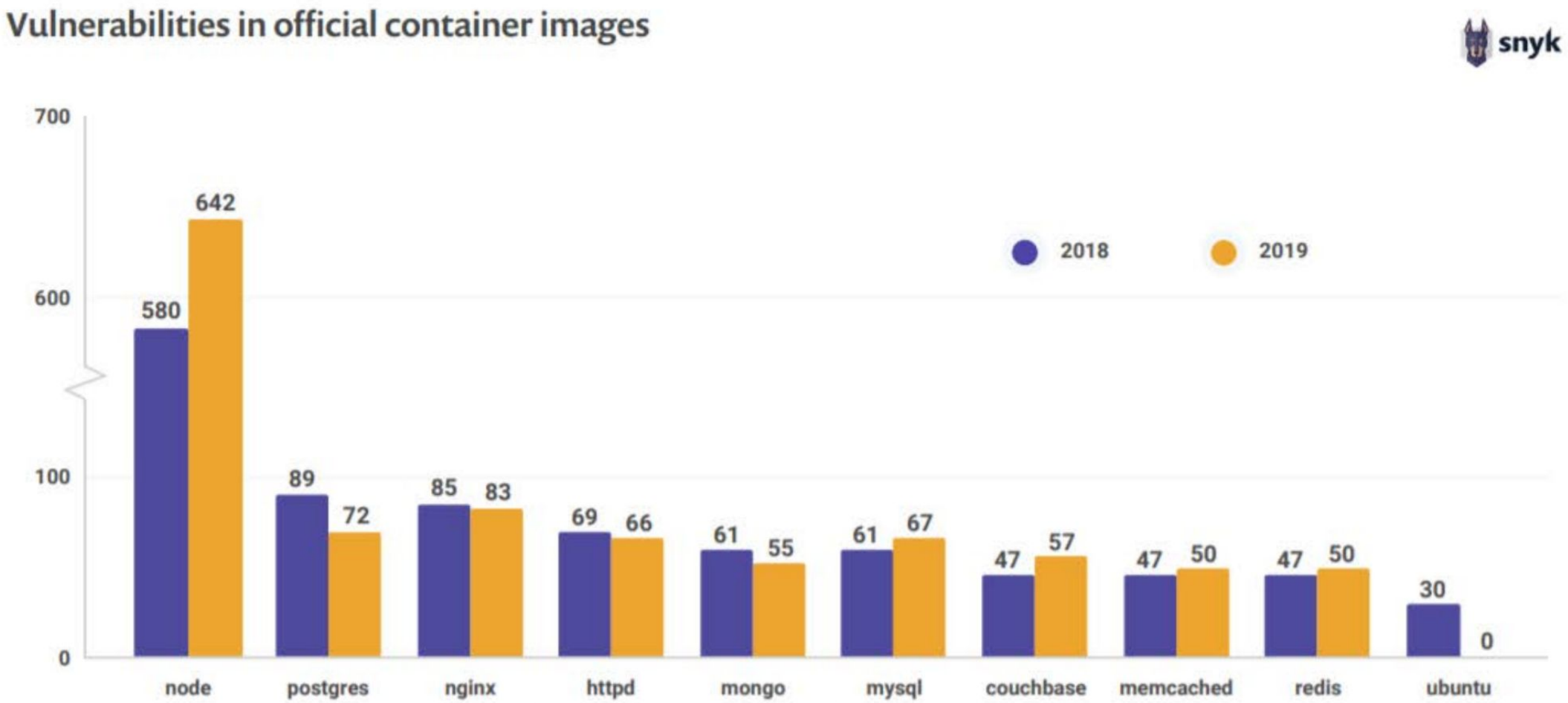




Open-source software

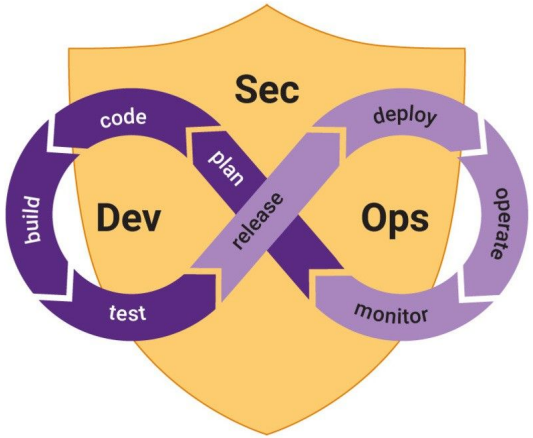


Threats in Open-source software



What is DevSecOps?

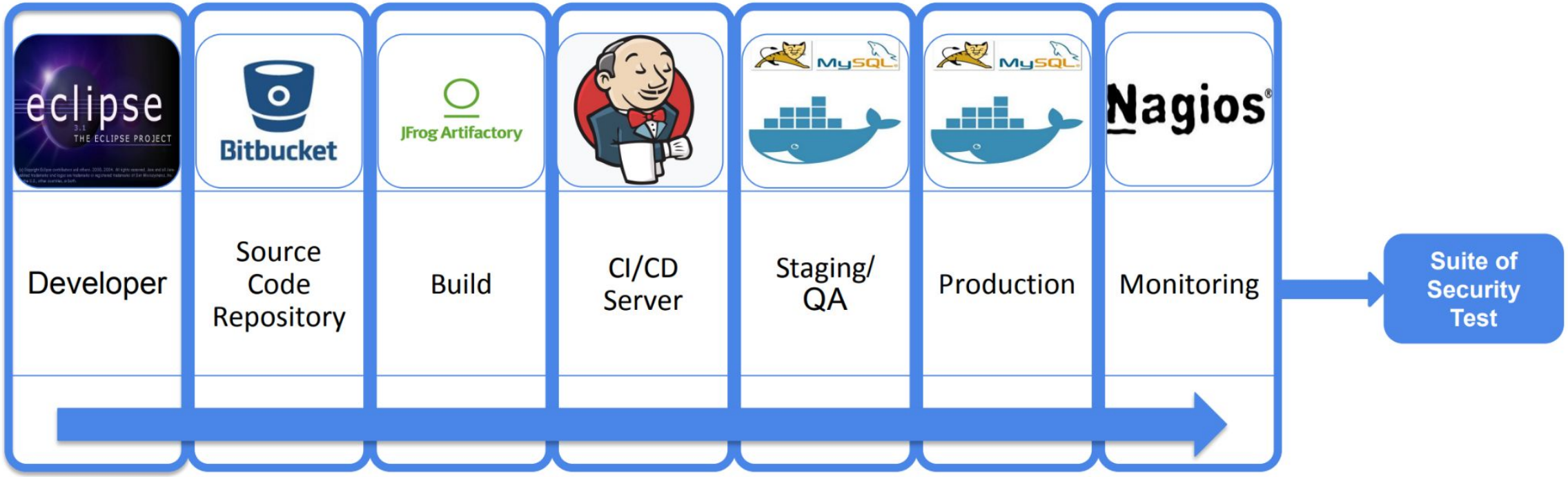
- DevSecOps stands for development, security, and operations. It's an approach to culture, automation, and platform design that integrates **security** as a shared responsibility throughout the entire IT lifecycle.



Why security in DevOps?

- The ability to deploy applications has improved in both scale and speed while security considerations are often overlooked in favor of meeting business demands quickly
- Given the reliance of applications to keep operations running; security in the development process cannot be an afterthought
- Application security must speed up to keep pace with operations

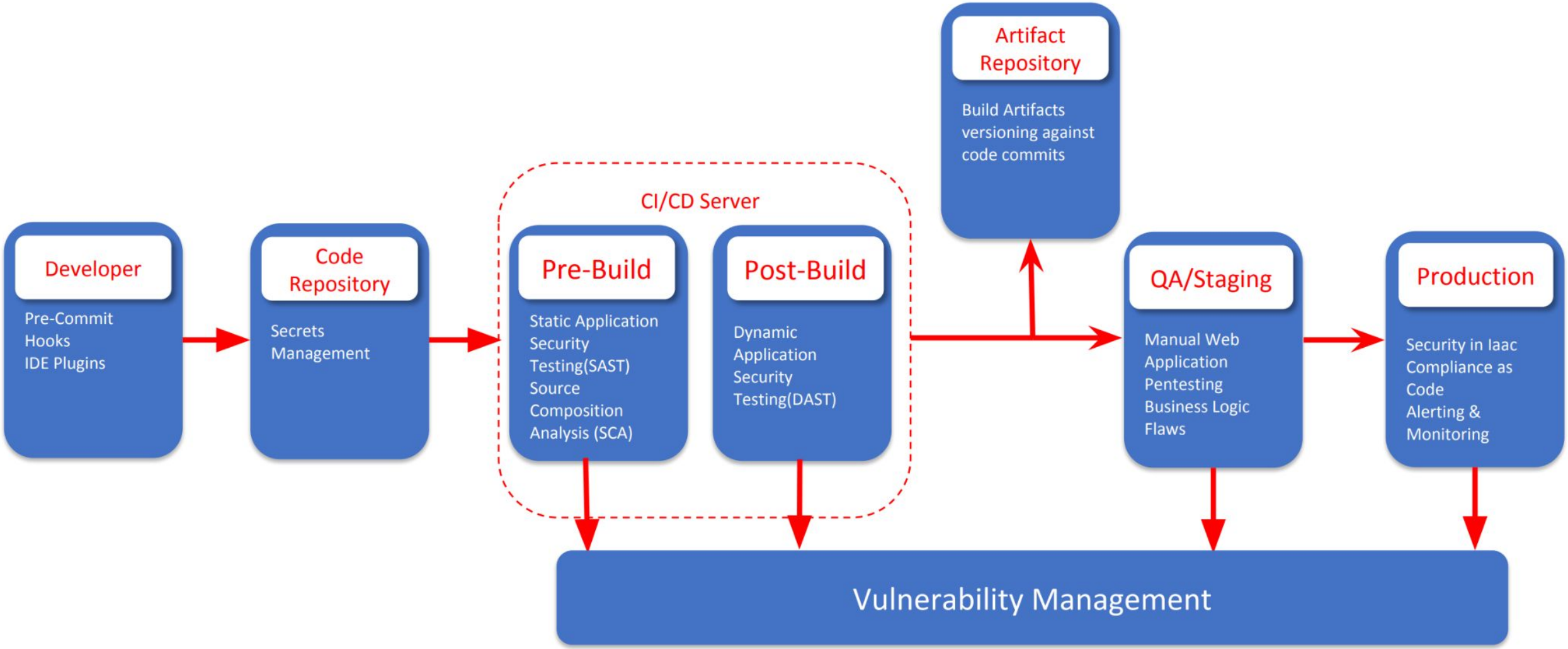
Why security in DevOps?



How can we bring security into DevOps?

- Tightly integrate security tools and processes throughout the DevOps pipeline
- Automate core security tasks by embedding security controls early on in the software development lifecycle
- Continuous monitoring and remediation of security defects across the application lifecycle including development and maintenance

Injecting Sec in DevOps



Pre-Commit Hooks

- Sensitive information such as the access keys, access tokens, SSH keys etc. are often erroneously leaked due to accidental git commits
- Pre-commit hooks can be installed on developer’s workstations to avoid the same
- Work on pure Regex-based approach for filtering sensitive data
- If developers want they can circumvent this step hence use it like a defense in depth but don't fully rely on it

Git hooks

Git hooks are scripts that run automatically every time a particular event occurs in a Git repository.

They let you customize Git's internal behavior and trigger customizable actions at key points in the development life cycle.

Git hooks

The built-in scripts are mostly shell and PERL scripts, but you can use any scripting language you like as long as it can be run as an executable.

```
#!/usr/bin/env python

import sys, os

commit_msg_filepath = sys.argv[1]
with open(commit_msg_filepath, 'w') as f:
    f.write("# Please include a useful commit message!")
```

Git hooks

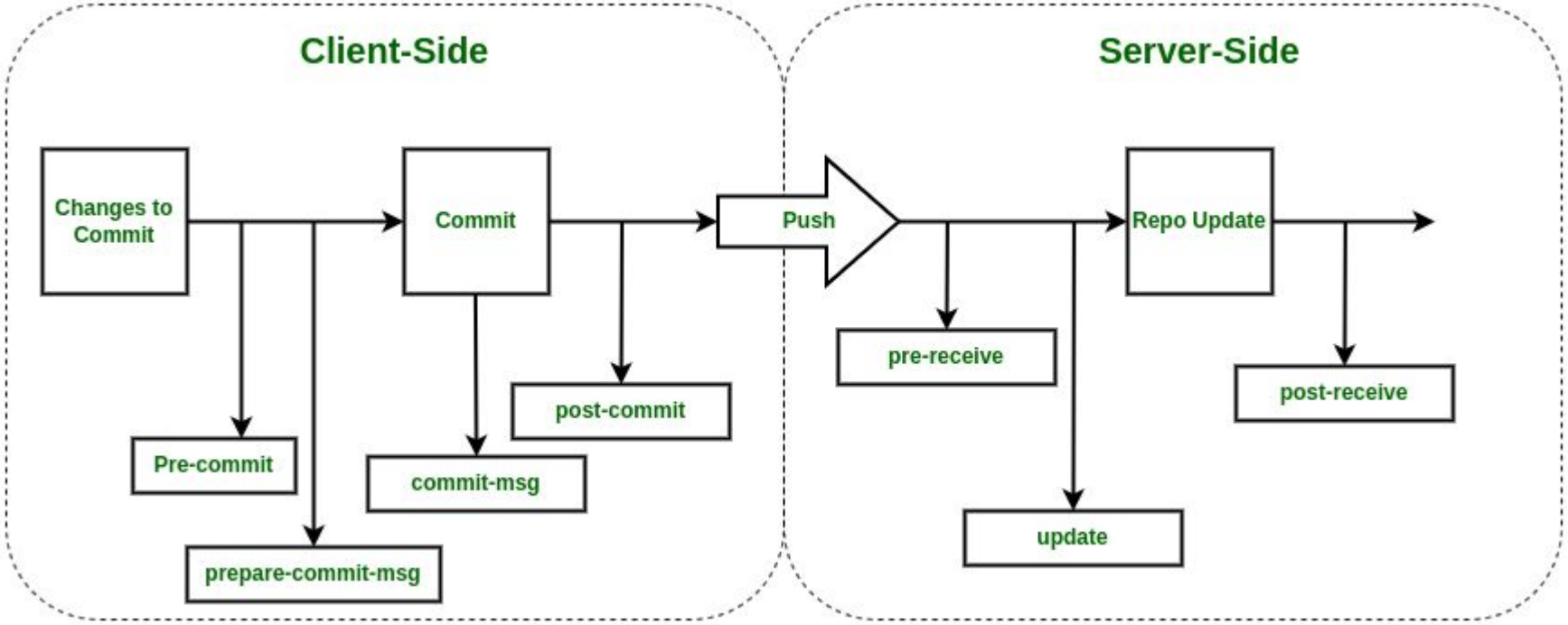
Hooks reside in the `.git/hooks` directory of every Git repository. Git automatically populates this directory with example scripts when you initialize a repository.

Git hooks

Hooks are local to any given Git repository, and they are not copied over to the new repository when you run `git clone`.

And, since hooks are local, they can be altered by anybody with access to the repository.

Local vs. Server-side hooks



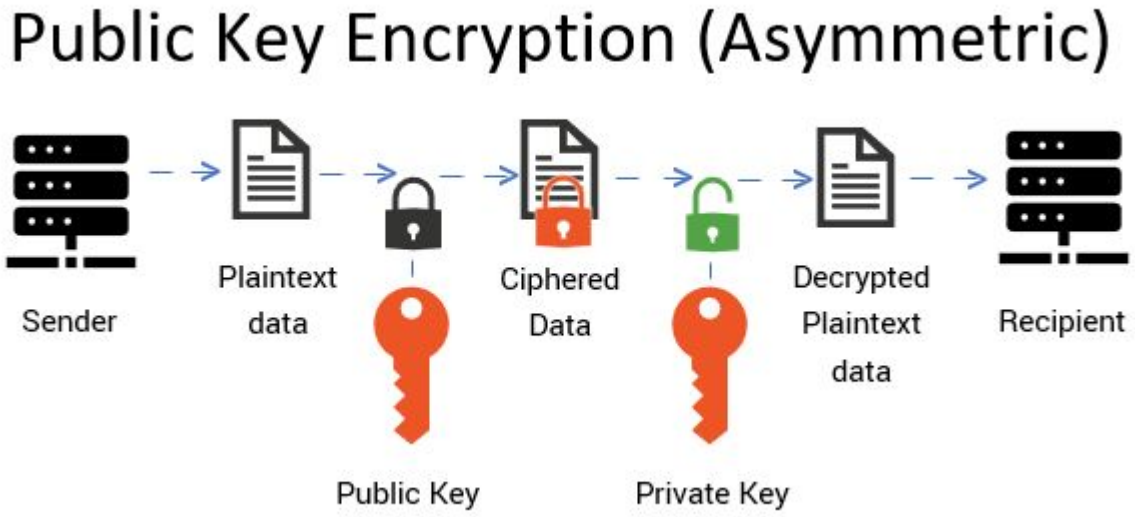
SSH

- SSH is a secure remote virtual terminal application
  - Provides encrypted communication between untrusted hosts over an insecure network
  - Assumes eavesdroppers can hear all communications between hosts
  - Provides different methods of authentication
  - Encrypts data exchanged between hosts

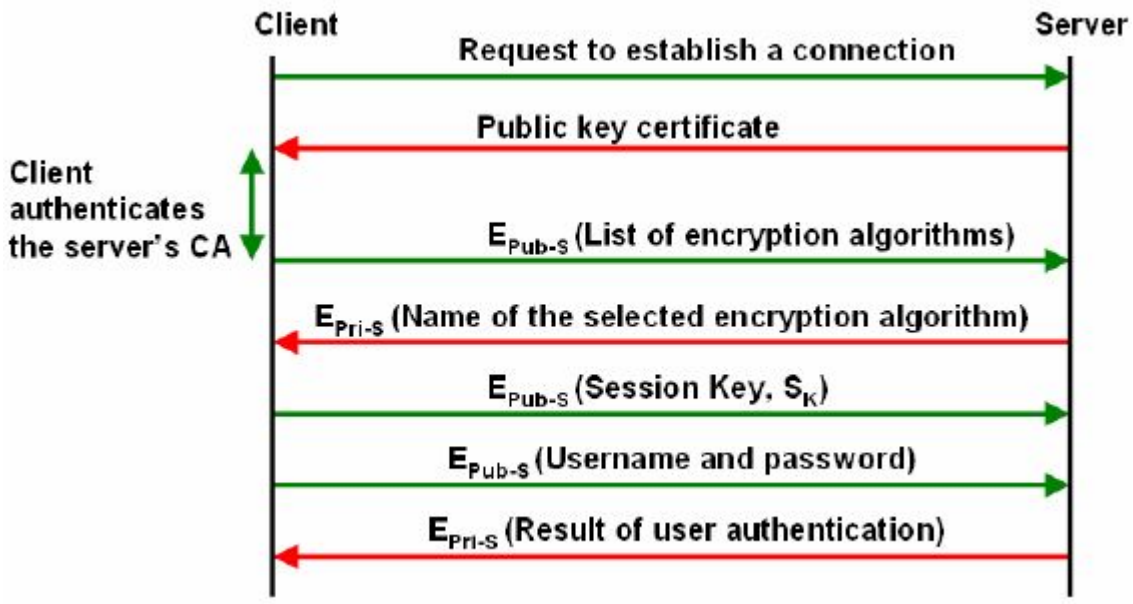
- Very popular and widely used
  - Not invulnerable!

SSH

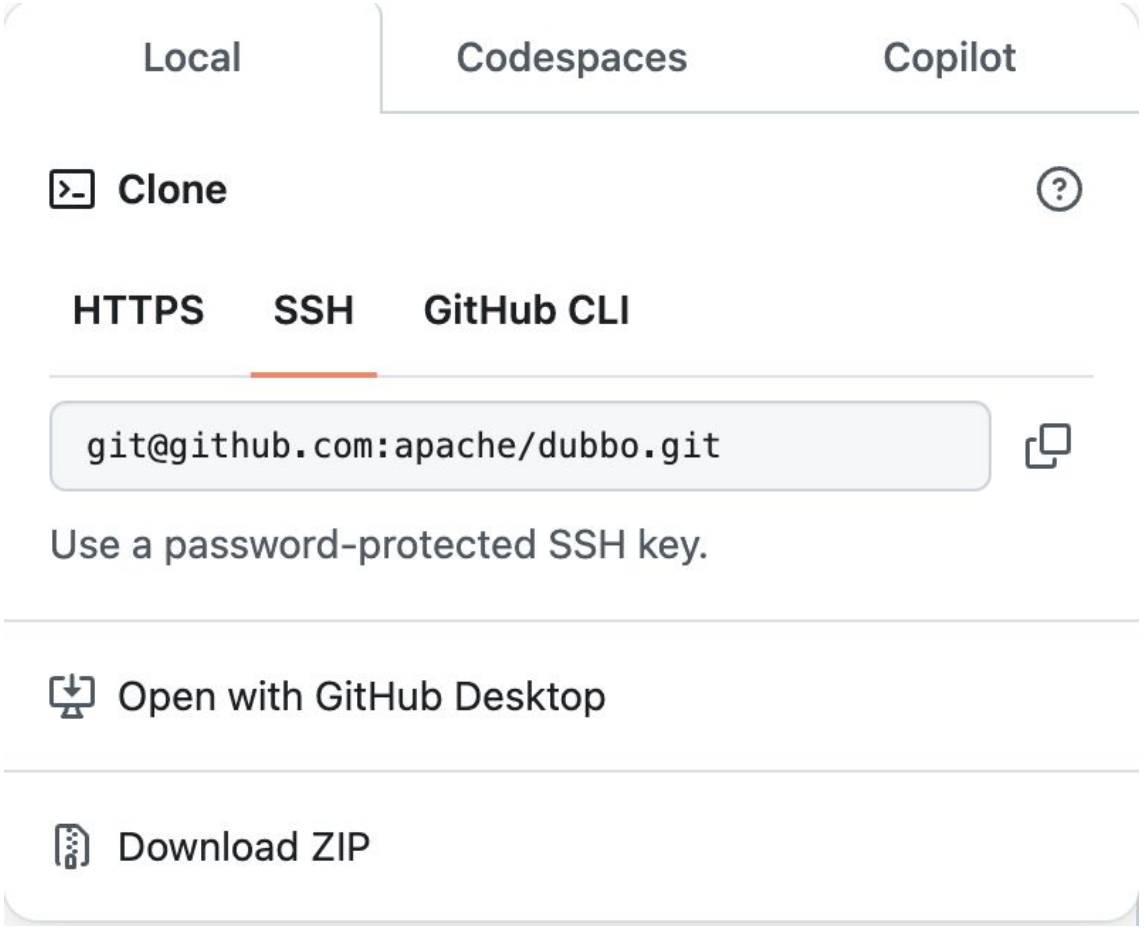
- Public key
  - shared openly and is primarily used to encrypt data or verify digital signatures.
  - used for encryption and signature verification.
- Private key
  - kept secret by its owner and is used to decrypt data encrypted with the public key or to create digital signatures.
  - used for decryption and signing.



SSH


Client
Server
Request to establish a connection
Public key certificate
Client authenticates the server's CA
E_{Pub-S} (List of encryption algorithms)
E_{Pri-S} (Name of the selected encryption algorithm)
E_{Pub-S} (Session Key, S_K)
E_{Pub-S} (Username and password)
E_{Pri-S} (Result of user authentication)

SSH


Local
Codespaces
Copilot
Clone
HTTPS
SSH
GitHub CLI
git@github.com:apache/dubbo.git
Use a password-protected SSH key.
Open with GitHub Desktop
Download ZIP

IDE Security Plugin

- IDE Plugin's provide quick actionable pointer to developers
- It is useful to stop silly security blunders
- Work on pure Regex-based approach
- If developers want they can circumvent this step hence use it like a defense in depth but don't fully rely on it

Secrets Management

- Often credentials are stored in config files
- Leakage can result in abuse scenario
- Secrets Management allows you to tokenize the information

Software Composition Analysis

- We don’t write software's, we build on frameworks
- Biggest portion of software is now third party libraries
- Major languages provide module managements
  - PIP, NPM, Gems, go get, perl cpan, php packager and more
- Software Composition Analysis performs checks to identify vulnerable/outdated 3rd party libraries

Static Analysis Security Testing

- White-box security testing using automated tools
- Useful for weeding out low-hanging fruits like SQL Injection, Cross-Site Scripting, insecure libraries etc
- Tool by default configured with generic setting, needs manual oversight for managing false-positives

Dynamic Analysis Security Testing

- Black/Grey-box security testing using automated tools
- SAST may not get full picture without application deployment
- DAST will help in picking out deployment specific issues
- Results from DAST and SAST can be compared to weed out false-positives
- Tools may need prior set of configuration settings to give good results

Security in Infrastructure as Code

- Infrastructure as a code (IaC) allows you to document and version control the infrastructure
- It also allows you to perform audit on the infrastructure
- Docker / Kubernetes (K8s) infrastructure relies on base images
- Environment is as secure as the base image
- Base images need to be minimal in nature and need to be assessed to identify inherited vulnerabilities

Compliance as Code

- Compliance could be industry standard (PCI DSS, HIPAA, SOX) or org specific
- Compliance is essentially a set of rules and hence can be converted into written test cases
- Having written code format this can again be version controlled

Vulnerability Management

- All the tools discussed above result in report fatigue
- Every tool has a different style of presentation
- A central dashboard is required to normalize the data
- Vulnerability Management System can then be integrated to bug tracking systems to allow devs to work on items

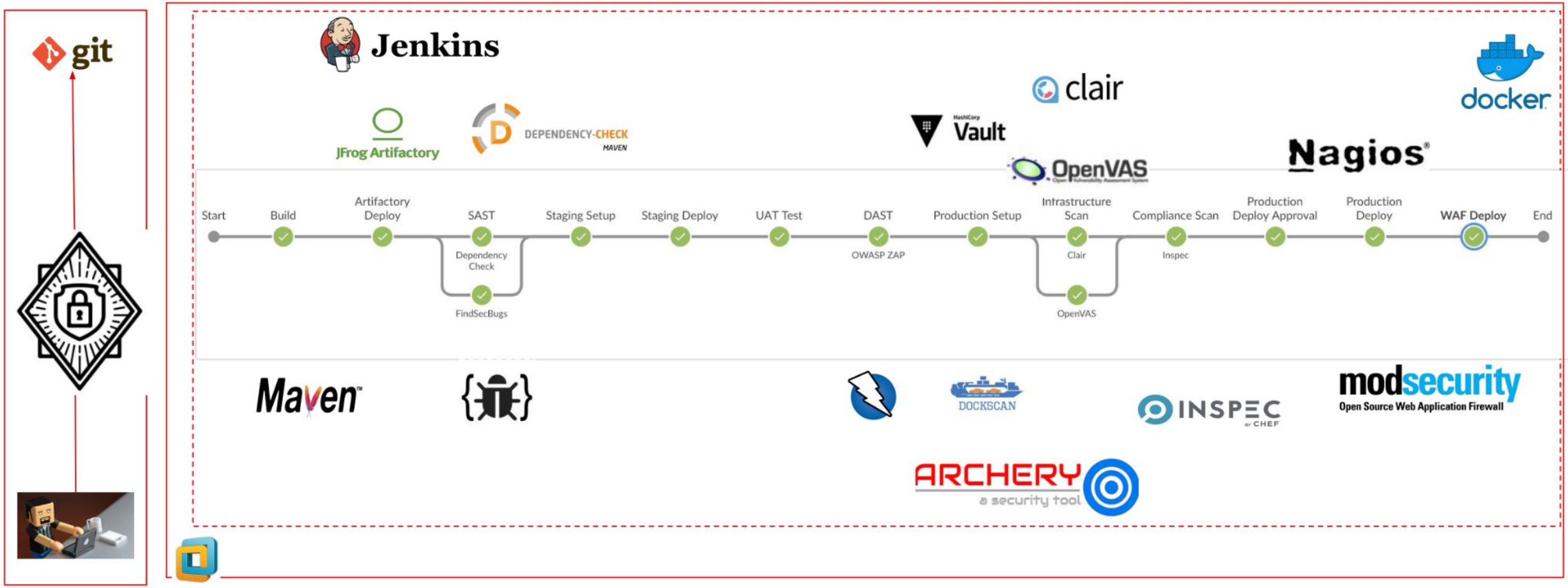
Alerting and Monitoring

- Monitoring is needed for two end goals
  - Understand if our security controls are effective
  - What and where we need to improve
- To test Security control effectiveness:
  - When did an attack occur
  - Was it blocked or not
  - What level of access was achieved
  - what data was bought in and bought out

Asset Monitoring

- With recent advancements assets now should include anything and everything where organization data resides
- With rapid development & provisioning the asset inventory can't be a static inventory
- We need to monitor the assets constantly both on premise and Cloud

A simplistic flow of DevSecOps Pipeline



Tools of The Trade

Threat Modelling Tools	  ThreatSpec Microsoft Threat Modeling Tool
Pre-Commit Hooks	   truffleHog Git Hound
Software Composition Analysis	  Retire.js
Static Analysis Security Testing (SAST)	  
IDE Plugins	 CAT.net   
Secret Management	 Keywhiz  Confidant

Tools of The Trade

Vulnerability Management	ARCHERY, JACKHAMMER, DEFECTDOJO
Dynamic Analysis Security Testing (DAST)	arachni, w3af, Wapiti
Security in Infrastructure as Code	OpenVAS (Open Vulnerability Assessment System), anchore, clair, DOCKSCAN, OpenSCAP
Compliance as Code	KitchenCI, INSPEC, DevSec Hardening Framework, Docker Bench for Security
WAF	modsecurity (Open Source Web Application Firewall), NAXSI

Language Specific Tools

Languages	Software Composition Analysis	Source Code Static Analysis
JAVA	DEPENDENCY-CHECK, ClearlyDefined, OSS Review Toolkit	sonarqube, graudit
PHP	OSS Review Toolkit, sonatype	sonarqube, graudit
Python	DEPENDENCY-CHECK, Safety, sonatype	Bandit, graudit, sonarqube
.NET	DotNET Retire, SafeNuGet, OSS Review Toolkit	PUMA SCAN, DotNet Security Guard
Ruby/Rails	OSS Review Toolkit, sonatype	Brakeman, sonarqube, graudit
Node JS	ClearlyDefined, OSS Review Toolkit	npm-check, NodeJsScan, sonarqube

Security Enablers

People

- Build relationships between teams, don't isolate
- Identify, nurture security conscious individuals
- Empower Dev / ops to deliver better and faster and secure, instead of blocking.
- Focus on solutions instead of blaming

Process

- Involve security from get-go (design or ideation phase)
- Fix by priority, don't attempt to fix it all
- Security Controls must be programmable and automated wherever possible
- DevSecOps Feedback process must be smooth and governed

Technology

- Templatize scripts/tools per language/platform
- Adopt security to devops flow don't expect others to adopt security
- Keep an eye out for simpler and better options and be pragmatic to test and use new tools

Thank you !